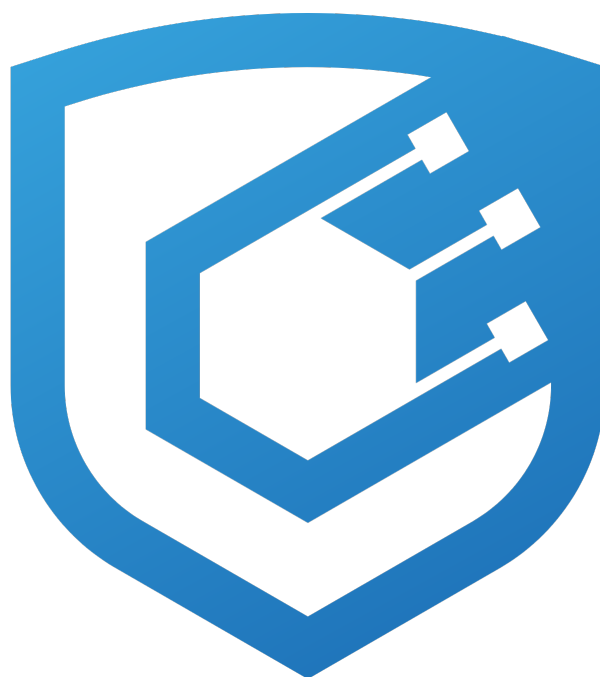




GreekFi Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Immeas](#)

[Alix40](#)

September 8, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	2
6.1	Centralization Risks	3
7	Findings	5
7.1	Medium Risk	5
7.1.1	Receipt::_payout lets attacker-chosen token granularity bypass all redemption fees	5
7.2	Low Risk	6
7.2.1	FactoryDeployer::deploy lets a front-runner invalidate a pre-mined vanity address	6
7.2.2	Receipt::setFee has no upper bound, letting the Factory owner take 100% of a live market's redemption proceeds	6
7.2.3	A single unrecoverable Receipt atom permanently disables sweep	7
7.2.4	Option::exercise uses the caller's live balance, unsolicited transfers can revert the call near expiry	8
7.3	Informational	9
7.3.1	DeployDeterministic::run does not verify the pinned Factory release	9
7.3.2	Public mapping accessors omit key and value names	9
7.3.3	Unused declarations remain in Factory and Option	9
7.3.4	Receipt::redeemFor reverts an entire pre-deadline batch after the consideration pool is exhausted	9
7.3.5	IOption documents the wrong pair-burn event	10
7.3.6	Receipt::sweep can clear accrued fee accounting without reporting the cleared amount	10
7.3.7	FlashExerciseKeeper is illustrative but is not explicitly marked unsafe for production use	10
7.3.8	IRceipt::redeem amount overload NatSpec misstates the revert for amounts above the caller's balance	11
7.3.9	Index key fields in Exercise, Expire, and Swept events	11
7.4	Gas Optimization	12
7.4.1	Receipt::collectFees loads the same fee accumulator twice	12
7.4.2	Factory::createOptions, createOptions2 copy read-only batch inputs into memory	12
7.4.3	Receipt::collectFees reads factory.owner() twice	12

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Greek.fi is a fully collateralized, oracle-free options protocol. Each market is an ERC20 pair of an `Option` (the long right to buy or sell collateral at a strike) and a `Receipt` (the short receipt for deposited collateral): users mint both sides 1:1 against collateral, an `Option` holder exercises by paying strike-denominated consideration to receive collateral, and a `Receipt` holder redeems consideration-first then collateral 1:1 on a purely time-gated schedule with no oracle ever consulted.

Each market is a non-upgradeable `Option` and `Receipt` pair created and registered by a `Factory` from fixed templates. The `Receipt` custodies the market's collateral and consideration, while the `Factory` is the shared allowance point: users approve it per token, and every registered `Receipt` can use that allowance for minting or exercising.

The owner cannot upgrade markets or directly sweep a funded `Receipt`, but can change its redemption fee without a cap or timelock, receive accrued fees, and sweep balances whenever its total supply is zero. Exercise and pair-burn remain fee-free.

5 Audit Scope

The audit scope was limited to:

```
contracts/Factory.sol
contracts/Option.sol
contracts/Permissions.sol
contracts/Receipt.sol
```

6 Executive Summary

Over the course of 3 days, the Cyfrin team conducted an audit on the [GreekFi](#) code provided by [GreekFi](#). The findings consist of 1 Medium & 4 Low severity issues with the remainder being informational and gas optimizations.

6.1 Centralization Risks

The factory owner sets the default fee applied to new markets (`Factory::setFee`, capped at 1000 bps) and is the sole payee of accrued protocol fees (`Receipt::collectFees`) and of residual balances swept only after a market is fully redeemed (`Receipt::sweep`).

The same owner can also reprice the redemption fee on any live market through `Receipt::setFee`, which is uncapped, so a high fee can skim or block that market's redemptions.

The owner cannot change a live market's collateral, strike, dates, tokens, or balances, and `Factory::renounceOwnership` is disabled, so the owner role can never be abandoned.

No party can pause, freeze, seize, claw back, or upgrade positions: every option and receipt is a clone of an immutable template with no pause switch or emergency withdrawal, so centralization is confined to the fee powers above.

Summary

Project Name	GreekFi
Repository	contracts
Commit	65f9843ecd9a...
Fix Commit	23bfee78157f...
Audit Timeline	Aug 31st - Sep 2nd, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	4
Informational	9
Gas Optimizations	3
Total Issues	17

Summary of Findings

[M-1] <code>Receipt::_payout</code> lets attacker-chosen token granularity bypass all redemption fees	Resolved
[L-1] <code>FactoryDeployer::deploy</code> lets a front-runner invalidate a pre-mined vanity address	Resolved
[L-2] <code>Receipt::setFee</code> has no upper bound, letting the Factory owner take 100% of a live market's redemption proceeds	Resolved
[L-3] A single unrecoverable Receipt atom permanently disables sweep	Resolved

[L-4] <code>Option::exercise</code> uses the caller's live balance, unsolicited transfers can revert the call near expiry	Resolved
[I-1] <code>DeployDeterministic::run</code> does not verify the pinned <code>Factory</code> release	Resolved
[I-2] Public mapping accessors omit key and value names	Resolved
[I-3] Unused declarations remain in <code>Factory</code> and <code>Option</code>	Resolved
[I-4] <code>Receipt::redeemFor</code> reverts an entire pre-deadline batch after the consideration pool is exhausted	Acknowledged
[I-5] <code>IOption</code> documents the wrong pair-burn event	Resolved
[I-6] <code>Receipt::sweep</code> can clear accrued fee accounting without reporting the cleared amount	Resolved
[I-7] <code>FlashExerciseKeeper</code> is illustrative but is not explicitly marked unsafe for production use	Resolved
[I-8] <code>IReceipt::redeem</code> amount overload <code>NatSpec</code> misstates the revert for amounts above the caller's balance	Resolved
[I-9] Index key fields in <code>Exercise</code> , <code>Expire</code> , and <code>Swept</code> events	Resolved
[G-1] <code>Receipt::collectFees</code> loads the same fee accumulator twice	Resolved
[G-2] <code>Factory::createOptions</code> , <code>createOptions2</code> copy read-only batch inputs into memory	Resolved
[G-3] <code>Receipt::collectFees</code> reads <code>factory.owner()</code> twice	Resolved

7 Findings

7.1 Medium Risk

7.1.1 `Receipt::_payout` lets attacker-chosen token granularity bypass all redemption fees

Description: `Receipt::_payout` calculates fees in the payout token's raw units and rounds down. Market creation is permissionless and imposes no minimum token decimals, so an attacker can deploy a zero-decimal wrapper whose single unit represents a valuable amount of an underlying asset.

For any fee below 100%, redeeming one unit produces a zero fee. Partial redemption lets the holder repeat one-unit redemptions, bypassing a fee that would be charged if the position were redeemed at once.

Ordinary rounding loses less than one raw unit per call. Here, the attacker controls the denomination and can make that raw unit economically valuable, turning marginal rounding into a complete fee bypass.

Impact: All redemption fees can be avoided in wrapper-denominated markets. The wrapper can be fully backed and redeemable, allowing genuine volume to be routed through the affected market. Fee loss is repeatable and scales with volume. Solvency and other holders' claims are unaffected.

Recommended Mitigation: Round nonzero fees up or carry fractional fees in accounting that cannot be reset through redemption splitting, token transfers, or new markets. Otherwise, restrict coarse token denominations or charge fees in a separate high-precision asset.

GreekFi: Fixed in [PR31](#)

Cyfrin: Verified. Receipt now rounds nonzero redemption fees up, so splitting redemptions or using a coarse token denomination cannot reduce the fee to zero.

7.2 Low Risk

7.2.1 FactoryDeployer::deploy lets a front-runner invalidate a pre-mined vanity address

Description: FactoryDeployer::deploy is permissionless, and owner is not included in the CREATE2 commitment. A searcher can copy a pending deployment's initCode and salt, deploy first with a different owner, and occupy the expected vanity address.

The official deployment script detects the collision and reverts, so the hostile deployment is not accepted as official.

Impact: An attacker can delay a new-chain rollout and force the team to mine a new vanity address. They cannot take control of a Factory accepted by the official deployment process. The impact is limited to deployment griefing.

Recommended Mitigation: Submit deployments privately and withhold salts until confirmation. Future versions should bind the owner into the CREATE2 commitment or require signed deployment authorization.

GreekFi: Fixed in [PR32](#)

Cyfrin: Verified. FactoryDeployer now binds the intended owner into the CREATE2 salt, so a caller using a different owner derives a different address.

7.2.2 Receipt::setFee has no upper bound, letting the Factory owner take 100% of a live market's redemption proceeds

Description: Receipt::setFee is callable by the Factory owner on any live market and stores feeBps_ after a caller check only:

```
/// @inheritdoc IReceipt
function setFee(uint64 feeBps_) external {
    if (msg.sender != address(factory) && msg.sender != factory.owner()) revert UnauthorizedCaller();
    feeBps = feeBps_;
    emit FeeUpdated(feeBps_);
}
```

Factory::setFee caps the creation-time default at 1000 bps, and Factory::createOption2 seeds each new market from that capped value via Receipt(receipt_).setFee(feeBps), but the per-market setter never re-applies the bound:

```
/// @inheritdoc IFactory
function setFee(uint64 bps) external onlyOwner {
    if (bps > 1000) revert InvalidValue();
    feeBps = bps;
    emit Fee(bps);
}
```

Receipt::_payout computes the fee and the net payout on both legs of every redemption:

```
function _payout(IERC20 token, address account, uint256 amount) internal returns (uint256 paid) {
    if (amount == 0) return 0;
    uint256 fee = Math.mulDiv(amount, feeBps, 10_000);
    if (fee > 0) feeAccrued[address(token)] += fee;
    paid = amount - fee;
    token.safeTransfer(account, paid);
}
```

At feeBps == 10_000 the redemption succeeds after Receipt::_redeem has burned the holder's receipts, pays 0, and credits the full gross amount to feeAccrued, which Receipt::collectFees pays to factory.owner(). Above 10_000 the subtraction underflows and every Receipt::redeem, redeemFor call on that market reverts until the owner lowers the fee. Pair-burn still works, but a writer who has sold their longs has no exit.

The IReceipt::setFee NatSpec acknowledges the missing cap:

```
/// @notice Set this market's redeem fee. Callable by the creating Factory during deployment or
///         by the current Factory owner afterward. No cap or timelock is applied; values above
```

```

/// 10000 bps make positive redemptions revert.
function setFee(uint64 feeBps_) external;

```

but the documented trust model says the opposite. The `IFactory::owner` `NatSpec` limits the owner's `setFee` reach to 10% and states it cannot touch a funded pool, and the README's "Trust boundaries" section and the Factory constructor note repeat the same guarantee:

```

/// @notice The {Ownable} owner. Its reach into the protocol: {setFee} ( 10 %, used as the
/// default during market initialization; {IReceipt.setFee} can reprice a live market),
/// {IReceipt.sweep} (gated on `totalSupply() == 0`) and receiving {IReceipt.collectFees}.
/// It cannot touch a live position or a funded pool. `renounceOwnership` is disabled
/// ({OwnershipNotRenounceable}) because an ownerless factory would strand every accrued
/// fee and sweepable balance in every Receipt it ever created.
/// @return The current owner; never `address(0)`.
function owner() external view returns (address);

```

The code implements the weaker claim.

Impact: The Factory owner can take 100% of a live, funded market's redemption proceeds, or halt its redemptions entirely, with a single fee update - contradicting the documented "cannot touch a funded pool" and " $\leq 10\%$ " owner-reach limits.

Recommended Mitigation: Enforce the same bound as `Factory::setFee` so the per-market rate can never exceed the documented 10%:

```

error InvalidFee(); // add to IReceipt

function setFee(uint64 feeBps_) external {
    if (msg.sender != address(factory) && msg.sender != factory.owner()) revert UnauthorizedCaller();
    if (feeBps_ > 1000) revert InvalidFee();
    feeBps = feeBps_;
    emit FeeUpdated(feeBps_);
}

```

GreekFi: Fixed in [PR39](#)

Cyfrin: Verified. Receipt now enforces the same 1,000 bps fee ceiling as Factory.

7.2.3 A single unrecoverable Receipt atom permanently disables sweep

Description: `Receipt::sweep` is the only way to recover tokens held above the amount required to back Receipt holders, but it reverts whenever `totalSupply() != 0`.

Receipts can be transferred to any address, including the Receipt contract itself. If a holder sends one Receipt atom there, the contract cannot redeem or burn it, so total supply can never return to zero. Every later sweep then reverts, even for balances that are provably unrelated to holder claims, such as donations, rounding residue, or accepted transfer over-delivery.

Impact: A griever can permanently disable surplus recovery for the cost of one Receipt atom and gas. The attacker cannot withdraw the stranded assets, and normal holder backing and redemptions remain solvent, so the impact is limited to a cheap denial of the rescue mechanism.

Recommended Mitigation: Allow the owner to sweep only the portion of the collateral or consideration balance that exceeds the amount required to back outstanding Receipts. Keep other tokens gated on zero supply and document that tokens sharing a balance ledger are unsupported.

GreekFi: Fixed in [PR47](#)

Cyfrin: Verified. Receipt now limits live-supply sweeps to balances above the collateral and consideration backing required by outstanding Receipts, so a stranded Receipt atom no longer disables recovery of fees, donations, or foreign tokens.

7.2.4 `Option::exercise` uses the caller's live balance, unsolicited transfers can revert the call near expiry

Description: The no-argument `Option::exercise` exercises the caller's full live balance. Anyone can transfer Options to the holder before the transaction. If the holder approved only enough consideration for their original balance, the added Options increase the pull in `Receipt::exercise`, causing the call to revert.

Impact: An attacker can delay exercise for the cost of the transferred Options. The holder can retry with `exercise(uint256)`, but a last-minute revert may make them miss the exercise window and lose the option's in-the-money value.

Recommended Mitigation: Document that the no-argument overload uses an externally inflatable balance and recommend `exercise(uint256)` for deadline-sensitive calls. Alternatively, make it best-effort by capping the exercise to the consideration the holder can currently spend:

```
uint256 optionBalance = balanceOf(msg.sender);
IERC20 cons = receipt.consideration();
uint256 budget = Math.min(
    cons.allowance(msg.sender, address(FACTORY)),
    cons.balanceOf(msg.sender)
);
uint256 amount = receipt.toConsideration(optionBalance, true) <= budget
    ? optionBalance
    : receipt.toCollateral(budget);
exerciseFor(msg.sender, amount);
```

Return the exercised amount and document that this overload may partially exercise.

GreekFi: Fixed in [PR46](#)

Cyfrin: Verified. The no-argument exercise documentation now warns that unsolicited transfers can increase the live balance and recommends the fixed-amount overload for deadline-sensitive calls.

7.3 Informational

7.3.1 `DeployDeterministic::run` does not verify the pinned `Factory` release

Description: `DeployDeterministic::run` hashes the locally compiled `Factory` creation code, predicts an address from that hash, deploys the same creation code, and then requires the returned address to equal the prediction. This equality confirms only that `CREATE2` behaved as expected. It holds for any locally compiled creation code and does not establish that the code matches the init-code hash or address pinned for the intended release.

Impact: An operator using the wrong source revision, compiler configuration, or linked library address can successfully deploy a `Factory` at an unintended address while every in-script assertion passes. Integrations configured for the release's canonical address will not use that deployment, requiring a corrected deployment and potentially causing operational confusion or cross-chain address drift.

Recommended Mitigation: Before broadcasting, require the locally computed init-code hash to equal a release-pinned hash and require the prediction to equal an explicitly supplied or release-pinned expected `Factory` address. Keep these expected values in a versioned deployment artifact rather than deriving both sides of the checks from the current local build.

GreekFi: Fixed in [PR32](#)

Cyfrin: Verified. The deployment script now checks the locally compiled `Factory` hash and predicted address against independently pinned release values before broadcasting.

7.3.2 Public mapping accessors omit key and value names

Description: The public `Factory::receipts`, `optionFor`, `permissions` and `Receipt::feeAccrued` mappings omit named key and value parameters. Their generated getters therefore expose less descriptive ABI metadata than the surrounding public interface.

Recommended Mitigation: Add descriptive key and value names to each mapping declaration, including both key levels of permissions.

GreekFi: Fixed in [PR35](#)

Cyfrin: Verified. The public mapping declarations now name their keys and return values, including both levels of permissions.

7.3.3 Unused declarations remain in `Factory` and `Option`

Description: `Factory::createOption` declares the named return variable `option_` but returns the result of `createOption2` directly without assigning that variable. Separately, `Option` imports `IERC20` without using the symbol. These declarations add noise without affecting the compiled behavior.

Recommended Mitigation: Make the `Factory::createOption` return value unnamed and remove the unused `IERC20` import from `Option`.

GreekFi: Fixed in [PR33](#)

Cyfrin: Verified. The unused `createOption` return name and unused `IERC20` import have been removed.

7.3.4 `Receipt::redeemFor` reverts an entire pre-deadline batch after the consideration pool is exhausted

Description: `Receipt::redeemFor` skips unauthorized and zero-balance holders, but forwards every other entry to `_redeem`. Before `exerciseDeadline`, collateral cannot be redeemed. Once earlier holders exhaust `consBacked`, the next non-zero holder therefore causes `ExerciseWindowOpen`, rolling back every earlier redemption in the transaction. This matches the documented atomic batch semantics: no holder loses a claim and the keeper can retry with a smaller batch, but the helper cannot provide best-effort progress and the failed attempt wastes gas.

Recommended Mitigation: If partial-progress batches are preferred, skip a holder when neither settlement leg is currently available. Keep the European pre-expiry revert and allow other `_redeem` failures to propagate:

```

function redeemFor(address[] calldata holders) external nonReentrant {
    for (uint256 i = 0; i < holders.length; i++) {
        address h = holders[i];
        if (notAuthorized(h, msg.sender, Perm.REDEEM)) continue;

        uint256 balance = balanceOf(h);
        if (balance == 0) continue;

        if (isEuro() && block.timestamp < expirationDate()) {
            revert BeforeExerciseWindow();
        }
        if (consBacked == 0 && block.timestamp <= exerciseDeadline()) continue;

        _redeem(h, balance);
    }
}

```

Update the `redeemFor` NatSpec to state that entries with no currently available settlement leg are skipped.

Greekfi: Accepted as intentional atomic batch behavior; no code change planned. `IRceipt.redeemFor` already states that only unauthorized and zero-balance holders are skipped and that any redemption revert rolls back the whole batch. `test/Sweep.t.sol::test_RedeemFor_RevertAbortsWholeBatch` pins the reported case.

7.3.5 `IOption` documents the wrong pair-burn event

Description: `IOption::transfer`, burn state that pair burns emit Redeemed from the Receipt and nothing from the Option. The implementation instead emits `PairBurned` from the Option, while `Receipt::burn` emits nothing.

Recommended Mitigation: Update the `IOption::transfer`, burn NatSpec to state that the Option emits `PairBurned` and the Receipt emits nothing. No implementation change is required.

GreekFi: Fixed in [PR46](#)

Cyfrin: Verified. The Option NatSpec now identifies `PairBurned` as the protocol event emitted by pair burns.

7.3.6 `Receipt::sweep` can clear accrued fee accounting without reporting the cleared amount

Description: `Receipt::sweep` resets `feeAccrued[token]` to its 1-wei floor before transferring the token balance. The `Swept` event reports only the total transfer, so off-chain accounting cannot determine how much accrued fee was cleared. On-chain accounting remains correct.

Recommended Mitigation: Emit the cleared fee amount, either in `Swept` or in a dedicated event.

GreekFi: Fixed in [PR43](#)

Cyfrin: Verified. Receipt now emits `FeeCleared` with the accrued-fee amount removed from accounting during a sweep.

7.3.7 `FlashExerciseKeeper` is illustrative but is not explicitly marked unsafe for production use

Description: The `FlashExerciseKeeper` snippet is labeled illustrative, and the surrounding NatSpec warns users to grant `Perm.EXERCISE` only to audited contracts. However, the example omits production checks and does not explicitly say it must not be deployed as-is. This is a documentation-hardening issue; the protocol contracts behave as specified.

Recommended Mitigation: Add a prominent NOT PRODUCTION READY warning stating that the snippet is pseudocode and must not be used in live deployments.

GreekFi: Fixed in [PR44](#)

Cyfrin: Verified. The keeper example is now explicitly marked as pseudocode that is not production-ready and must not be deployed as written.

7.3.8 `IRceipt::redeem` amount overload NatSpec misstates the revert for amounts above the caller's balance

Description: `IRceipt::redeem(uint256)` documents that before `exerciseDeadline` an amount above the caller's balance is not an error, and that after the deadline it reverts `InsufficientPool` rather than `ERC20InsufficientBalance`. Neither holds once a market has more than one holder: `Receipt::_redeem` caps `amount_` against the global `consBacked` and never against the caller's balance, so it reaches `_burn` and reverts `ERC20InsufficientBalance` in both cases. The `_redeem` dev-comment already hedges the opposite way, so the two doc sites disagree.

Impact: The `redeem(uint256)` NatSpec is not correct on either side of the deadline, and no funds are affected.

Recommended Mitigation: Correct the `IRceipt::redeem(uint256)` NatSpec to match the implementation, or clamp `amount_` to the caller's balance in `Receipt::_redeem`.

GreekFi: Fixed in [PR45](#)

Cyfrin: Verified. The `redeem` amount NatSpec now describes the actual capped pre-deadline burn, full post-deadline burn, and possible revert ordering.

7.3.9 Index key fields in Exercise, Expire, and Swept events

Description: `Exercise` and `Expire` do not index either account, while `Swept` indexes only `caller`. Indexers cannot filter these logs by holder, keeper, token, or recipient without fetching and decoding every event from each clone.

Impact: No contract behavior or funds are affected. Monitoring and accounting are less efficient.

Recommended Mitigation: Index `caller` and `holder` in `Exercise`; index either account field in `Expire`, since both are always equal; and index `token` and `recipient` in `Swept`.

GreekFi: Fixed in [PR43](#)

Cyfrin: Verified. `Exercise`, `Expire`, and `Swept` now index the relevant caller, holder, recipient, and token fields.

7.4 Gas Optimization

7.4.1 Receipt::collectFees loads the same fee accumulator twice

Description: Receipt::collectFees reads feeAccrued[token] once for the zero-value guard and again immediately afterward to calculate the collectible amount. There is no intervening write or external call, and optimized IR retains both mapping storage loads.

Recommended Mitigation: Load feeAccrued[token] into a local variable, use the local for the zero-value guard, and derive the collectible amount from the same local.

GreekFi: Fixed in [PR37](#)

Cyfrin: Verified. Receipt now loads the fee accumulator once and reuses the cached value.

7.4.2 Factory::createOptions, createOptions2 copy read-only batch inputs into memory

Description: Factory::createOptions, createOptions2 are external entry points whose dynamic array parameters are read-only, but the arrays are declared as memory. The ABI decoder therefore copies the complete arrays from calldata before the loops use them. The cost grows with every batched market and salt.

Recommended Mitigation: Declare the external batch parameters as calldata and refactor the shared creation logic so each entry can be consumed without copying the entire arrays into memory. Ensure the internal helper boundary also accepts calldata-compatible input; otherwise the copy is merely moved into each loop iteration.

GreekFi: Fixed in [PR33](#)

Cyfrin: Verified. Factory now keeps read-only creation parameters and batch arrays in calldata through the shared creation path.

7.4.3 Receipt::collectFees reads factory.owner() twice

Description: Receipt::collectFees makes two external calls to factory.owner(), once as the transfer recipient and once for the Fee event:

```
function collectFees(address token) external nonReentrant {
    if (feeAccrued[token] == 0) return; // virgin slot (leg never accrued) - guards the c - 1 below
    uint256 c = feeAccrued[token] - 1; // strip the 1-wei floor + the real collectible
    if (c == 0) return; // only the floor remained; nothing to collect
    feeAccrued[token] = 1;
    IERC20(token).safeTransfer(factory.owner(), c);
    emit Fee(token, factory.owner(), c);
}
```

One call and a local suffices, and it also guarantees the event names the address that received the tokens.

Recommended Mitigation:

```
address to = factory.owner();
IERC20(token).safeTransfer(to, c);
emit Fee(token, to, c);
```

GreekFi: Fixed in [PR37](#)

Cyfrin: Verified. Receipt now reads the Factory owner once and reuses that address for both the transfer and event.